

3D RPG Demo 紹介

制作期間：約4か月

授業外の個人制作として、企画、実装、調整、コード整理、提出用資料作成まで行いました。

概要

本作は、DxLib と C++ で制作した 3D アクション RPG デモです。

プレイヤーは Knight または Wizard を選択し、未選択のキャラクターは AI 相棒として戦闘に参加します。

デモは、Mutant との近接戦闘を中心とした `MutantScene` と、魔法弾を使う Vampire とのボス戦を中心とした `VampireScene` で構成されています。

主な展示内容

- キャラクター選択と、選ばれなかったキャラクターの AI 制御
- Knight の近接攻撃、ガード、カウンター、回避
- Wizard の魔法攻撃、魔法ビーム、回復、範囲魔法
- Mutant の通常攻撃、突進、回転攻撃、ジャンプ攻撃
- Vampire の魔法弾、誘導弾、テレポートを含む戦闘演出
- キーボード・マウスとゲームパッドの入力切り替え
- 使用中の入力デバイスに応じた UI 表示
- 音量、画面、操作割り当てを変更できる設定画面

技術的な特徴

- C++ によるシーン管理、キャラクター制御、戦闘判定の実装
- `SceneBase` を基底としたシーン遷移構造
- `CharBase` を基底とした Player / Enemy の共通化
- `InputManager` によるキーボード、マウス、ゲームパッド入力の統合
- `GameSettings` による設定値と入力割り当ての管理
- `ResourceManager` による画像、モデル、サウンドリソース管理
- `AttackEffect` による攻撃、魔法、ヒット、回復エフェクト管理
- `ThreatComponent` による Enemy のターゲット選択補助

コード構造

コード構造は、添付のクラス図 `クラス図.png` から確認できます。

Scene

`mutant\_scene.cpp`

MutantScene の基本更新、描画、シーン全体の流れを管理します。

`mutant\_scene\_combat.cpp`

プレイヤー攻撃、敵攻撃、ダメージ処理を担当します。

`mutant\_scene\_enemy.cpp`

Mutant 系 Enemy の生成と更新を担当します。

`mutant\_scene\_marker.cpp`

VampireScene への転送マーカーやテレポート演出を担当します。

`vampire\_scene.cpp`

VampireScene の基本更新、描画、ボス戦全体の流れを管理します。

`vampire\_scene\_combat.cpp`

Vampire 戦での攻撃、魔法、ダメージ処理を担当します。

`setting\_scene.cpp` / `setting\_scene\_navigation.cpp` / `setting\_scene\_binding.cpp` /

`setting\_scene\_draw.cpp`

設定画面を基本処理、ページ移動、入力割り当て、UI 描画に分けています。

Character / Combat

`player.cpp` / `player\_movement.cpp` / `player\_combat.cpp`

Player の基本処理、移動、戦闘処理を分けています。

`enemy.cpp` / `enemy\_ai\_controller.cpp` / `enemy\_attack.cpp` / `enemy\_collision.cpp` /

`enemy\_jump\_attack.cpp` / `enemy\_reaction.cpp`

Enemy の基本処理、AI、攻撃、当たり判定、ジャンプ攻撃、被弾リアクションを分けています。

`magic\_projectile\_enemy.cpp`

Vampire 系ボスの魔法弾、誘導、着弾、テレポート処理を管理します。

`attack\_effect.cpp` / `attack\_effect\_spawn.cpp` / `attack\_effect\_dx11.cpp` /

`attack\_effect\_dxlib.cpp`

エフェクトの生成、更新、DirectX 11 描画、DxLib fallback 描画を分けています。

実装で意識した点

1 つの cpp に処理が集中しすぎないように、戦闘、AI、判定、描画を機能単位で分割しました。

UI や戦闘の入力は、キーボード・マウスとゲームパッドの両方を想定しています。

エフェクト描画では DirectX 11 を使用し、使用できない場合は DxLib 描画へフォールバックします。

設定画面では入力割り当ての競合を検出し、無効な設定のまま戻らないようにしています。

コメントは Doxygen 形式を中心に追加し、クラスや関数の責務が追いやすようにしています。

今後の改善候補

Enemy と MagicProjectileEnemy の共通部分を整理し、敵キャラクター拡張時の見通しを改善する。

設定画面の UI 部品を小さな描画 helper として整理し、ページ追加をしやすいようにする。